23 September 2025 -> 6 February 2026

AP155 Problem Set 1

Elisha Gabriel B. Agoot

In [3]:
```
import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
```

Problem 4.17

Use **LU decomposition** (without and with pivoting) for:

```
A = np.array([4., 4, 8, 4, 4, 5, 3, 7, 8, \
3, 9, 9, 4, 7, 9, 5]).reshape(4,4)
bs = np.array([1., 2, 3, 4])
```

- [x] Does pivoting help in any way?
- [x] What are your conclusions about this matrix?

You sometimes hear the advice that, since the problem arises from one of the U_{ii}'s being zero, you should replace it with a small number, say 10^{-20}.

- [ ] Does this work? Explain.

In [303…
```
A = np.array([4.,4,8,4, \
              4,5,3,7, \
              8,3,9,9, \
              4,7,9,5]).reshape(4,4)
bs = np.array([1.,2,3,4])
'''
    where due to the declaration of their respective first elements,
    all other elements are of the type float64.
'''
```

Out[303…
```
'\n    where due to the declaration of their respective first elements,\
n    all other elements are of the type float64.\n'
```

Note that the augmented matrix $\bm{A}|\bm{b}$ is of the form

$$\bm{A}|\bm{b} = \left(\begin{array}{cccc|c} 4 & 4 & 8 & 4 & 1 \\ 4 & 5 & 3 & 7 & 2 \\ 8 & 3 & 9 & 9 & 3 \\ 4 & 7 & 9 & 5 & 4 \end{array}\right)$$

Using elementary row operations, we try to decompose it without pivoting first.

$$\bm{A}|\bm{b} = \left(\begin{array}{cccc|c} 4 & 4 & 8 & 4 & 1 \\ 4 & 5 & 3 & 7 & 2 \\ 8 & 3 & 9 & 9 & 3 \\ 4 & 7 & 9 & 5 & 4 \end{array}\right)$$

$$\begin{aligned} R_2' &= R_2 - R_1 \\ R_3' &= R_3 - 2R_1 \\ R_4' &= R_4 - R_1 \end{aligned} \rightarrow \left(\begin{array}{cccc|c} 4 & 4 & 8 & 4 & 1 \\ 0 & 1 & -5 & 3 & 1 \\ 0 & -5 & -7 & 1 & 1 \\ 0 & 3 & 1 & 1 & 3 \end{array}\right)$$

$$\begin{aligned} R_3' &= R_3 + 5R_2 \\ R_4' &= R_4 - 3R_2 \end{aligned} \rightarrow \left(\begin{array}{cccc|c} 4 & 4 & 8 & 4 & 1 \\ 0 & 1 & 5 & 3 & 1 \\ 0 & 0 & -31 & 16 & 6 \\ 0 & 0 & 16 & 8 & 0 \end{array}\right)$$

$$R_4' = R_4 - \frac{1}{2}R_3 \rightarrow \left(\begin{array}{cccc|c} 4 & 4 & 8 & 4 & 1 \\ 0 & 1 & 5 & 3 & 1 \\ 0 & 0 & -31 & 16 & 6 \\ 0 & 0 & 0 & 0 & 3 \end{array}\right)$$

Now, there is a line of thoughts/concepts in Math 40 (Linear Algebra) that connects some properties of matrices to their "solvability," so to speak. Notably, this set of statements is equivalent:

1. A matrix $\bm{A}$ is nonsingular if and only if there exists an inverse $\bm{A}^{-1}$.
2. The linear equation $\bm{A}\bm{x} = \bm{b}$ has a unique solution $\bm{x} = \bm{A}^{-1}\bm{b}$ for any $\bm{b}$.
3. $\bm{A}\bm{x} = \bm{0}$ has the trivial solution $\bm{x} = \bm{0}_n$, where $\bm{0}$ is the null vector of length n.
4. $\text{RREF}(\bm{A}) = \bm{I}_n$, where $\bm{I}$ is the identity matrix of dimension n.
5. The matrix $\bm{A}$ is a product of elementary matrices.
6. The matrix's determinant exists, such that it is nonzero.

The fourth statement refers to what is known as the Reduced Row Echelon Form of a matrix, which is derived if we continue applying row operations such that the resulting matrix has its pivot points (in our case, A_{ii}) **as the only nonzero value in its column.**

Due to the presence of the last row being completely 0 in our decomposition of $\bm{A}$, then its RREF can never become an identity matrix (contains one at all elements of the principal diagonal). It then follows that our given matrix A, however square:

- (from 1) is a singular matrix, therefore its inverse does not exist;
- (from 2) does not have a unique solution for any vector $\bm{b}$: either infinite solutions or no solution at all; and
- (from 6) has a determinant $\det(A) = 0$.

Furthermore, the nature of last row reveals that this is an *inconsistent* system, since the equivalent equation would imply that as the coefficients of all elements of $\bm{x}$ are 0, their sum would somehow produce a non-zero number.

> So specifically, the linear equation $\bm{A}x = \bm{b}$ with our given $\bm{A}$ will have no solution at all for any given $\bm{b}$ of compatible dimension. Pivoting would not help in any way due to the existence of a dependent (and inconsistent) row.

However, this does not yet count out if the matrix itself is $\bm{LU}$-decomposable or not. We can extend our notion by considering the nature of the matrix reduction: even in the latest result of our row operations, we see that we have reduced our matrix into an upper triangular matrix. Linear algebra posits that all previous row operations are *actually* matrices themselves that have been "factored out" of our augmented matrix. It is then a good guess that if all of those row operation matrices are multiplied together, then they should form a lower triangular matrix.

In [304…
```
x = sp.linalg.solve(A, bs)
```

```
/opt/anaconda3/envs/pisika/lib/python3.13/site-packages/scipy/_lib/_util.p
y:1226: LinAlgWarning: Ill-conditioned matrix (rcond=6.56289e-18): result
may not be accurate.
  return f(*arrays, *other_args, **kwargs)
```

In [305…
```
np_det_A = np.linalg.det(A)
sp_det_A = sp.linalg.det(A)

print(f'NumPy det(A) = {np_det_A}')
print(f'SciPy det(A) = {sp_det_A}')
print(f'xs = {x}')
A @ x == bs
```

```
NumPy det(A) = 0.0
SciPy det(A) = 0.0
xs = [-4.50359963e+15 -1.50119988e+15  1.50119988e+15  3.00239975e+15]
```

Out[305…
```
array([False, False, False, False])
```

The $\bm{x}$ matrix containing entirely large numbers is uncharacteristic of our given $\bm{A}$ and $\bm{b}$, thus it can be attributed to floating-point errors. The `linalg` package itself tells that our matrix is ill-conditioned. Obviously, plugging this resultant $\bm{x}$ into our linear equation will not produce our $\bm{b}$ by orders of magnitude.

Although with it, the evaluation of the determinant is in agreement with Statement 6, and even colludes with my guess of our matrix to still be LU-decomposable.

According to the SciPy documentation of the determinant function `det()`, it is computed by performing an LU factorization of the input, using the `getrf` routine of the LAPACK (Linear Algebra Package). A resulting determinant that's consistent with theory and an implicit success on LU factorization of $\bm{A}$ only leads to assume that an LU factorization does indeed exist for our $\bm{A}$.

In [306…
```
p, l, u = sp.linalg.lu(A)

print(f'P = \n {p}')
print(f'L = \n {l}')
print(f'U = \n {u} \n')
```

```
P =
 [[0. 0. 0. 1.]
 [0. 0. 1. 0.]
 [1. 0. 0. 0.]
 [0. 1. 0. 0.]]
L =
 [[ 1.          0.          0.          0.        ]
 [ 0.5         1.          0.          0.        ]
 [ 0.5         0.63636364  1.          0.        ]
 [ 0.5         0.45454545 -0.33333333  1.        ]]
U =
 [[ 8.          3.          9.          9.        ]
 [ 0.          5.5         4.5         0.5       ]
 [ 0.          0.         -4.36363636  2.18181818]
 [ 0.          0.          0.          0.        ]]
```

And here it is, above, in discrete form.

If we do follow the advice to make the fourth diagonal number to a *really* small number close to zero, here's what happens.

In [358…
```
u[-1, -1] = 1E-20
A_new = p @ l @ u

x_new = sp.linalg.solve(A_new, bs)
delta = A_new - A
print(f' x_new = \n {x_new} \n\n A_new - A = \n{delta}')
```

```
x_new =
 [ 3.37769972e+15  1.12589991e+15 -1.12589991e+15 -2.25179981e+15]

A_new - A =
[[0.0000000e+00 0.0000000e+00 0.0000000e+00 8.8817842e-16]
 [0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00]
 [0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00]
 [0.0000000e+00 0.0000000e+00 0.0000000e+00 0.0000000e+00]]
```

```
/opt/anaconda3/envs/pisika/lib/python3.13/site-packages/scipy/_lib/_util.p
y:1226: LinAlgWarning: Ill-conditioned matrix (rcond=8.75053e-18): result
may not be accurate.
  return f(*arrays, *other_args, **kwargs)
```

In [362…
```
#   I use the word sensitivity below because
#   the degree of change that we do introduce *does*
#   change the resultant x vector into something more sensible.

#   Suppose we increase the size of our change to
u[-1, -1] = 1E-10
#   The x-vector's components actually drops fast in value.
#       It should follow that if we set the above value to the
```

```
#       same magnitude as the other components of A, then we
#       would probably reach a solution BUT keeping in mind that
#       it is not the same matrix A anymore.

A_new = p @ l @ u

x_new = sp.linalg.solve(A_new, bs)
delta = A_new - A
print(f' x_new = \n {x_new} \n\n A_new - A = \n{delta}')
```

```
x_new =
[ 2.99997311e+10  9.99991036e+09 -9.99991036e+09 -1.99998207e+10]

A_new - A =
[[0.00000000e+00 0.00000000e+00 0.00000000e+00 1.00000896e-10]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]
 [0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00]]
```

The resultant $\bm{x}$ vector is still composed of uncharacteristically large numbers. The most sound conclusion that we can derive from this is that we are seeing seeing some *sensitivity* of the original matrix to small changes of value, but none of that will lead to any solution because it ***is*** ill-conditioned in the first place due to underlying mathematics.

Alternatively, here's another solution if we assume from the start that $\bm{A}$ is indeed LU decomposable.

In [363…
```
lu, piv = sp.linalg.lu_factor(A)
x_alt = sp.linalg.lu_solve((lu, piv), bs)

lu, piv, x_alt
```

```
/opt/anaconda3/envs/pisika/lib/python3.13/site-packages/scipy/_lib/_util.p
y:1226: LinAlgWarning: Diagonal number 4 is exactly zero. Singular matrix.
  return f(*arrays, *other_args, **kwargs)
```

Out[363…
```
(array([[ 8.        ,  3.        ,  9.        ,  9.        ],
        [ 0.5       ,  5.5       ,  4.5       ,  0.5       ],
        [ 0.5       ,  0.63636364, -4.36363636,  2.18181818],
        [ 0.5       ,  0.45454545, -0.33333333,  0.        ]]),
 array([2, 3, 3, 3], dtype=int32),
 array([ nan,  inf, -inf, -inf]))
```

In [342…
```
print(f'Is x     a valid solution? {np.allclose(A @ x - bs, np.zeros((4,)
print(f'Is x_new a valid solution? {np.allclose(A @ x_new - bs, np.zeros(
print(f'Is x_alt a valid solution? No as well, seeing that it\'s not even
```

```
Is x     a valid solution? False
Is x_new a valid solution? False
Is x_alt a valid solution? No as well, seeing that it's not even comprised
of numbers.
```

Problem 4.18

We will now employ the Jacobi iterative method to solve a linear system of equations, for the case where the coefficient matrix is sparse; we won't need to explicitly store $\bm{A}$.

Focus on the following cyclic tridiagonal matrix:

$$(\mathbf{A}|\mathbf{b}) = \left(\begin{array}{cccccccccc|c} 4 & -1 & 0 & 0 & \dots & 0 & 0 & 0 & -1 & 1 \\ -1 & 4 & -1 & 0 & \dots & 0 & 0 & 0 & 0 & 1 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & 0 & -1 & 4 & -1 & 1 \\ -1 & 0 & 0 & 0 & \dots & 0 & 0 & -1 & 4 & 1 \end{array}\right) \tag{4.241}$$

Something similar appears when you study differential equations with periodic boundary conditions (Note that $\bm{A}$ is "almost a tridiagonal matrix").

For this problem, the Jacobi iterative method turns into:

$$\begin{aligned} x_0^{(k)} &= \left(1 + x_1^{(k-1)} + x_{n-1}^{(k-1)}\right)\frac{1}{4} \\ x_i^{(k)} &= \left(1 + x_{i-1}^{(k-1)} + x_{i+1}^{(k-1)}\right)\frac{1}{4}, \quad i \in \{1, \dots, n-2\} \\ x_{n-1}^{(k)} &= \left(1 + x_0^{(k-1)} + x_{n-2}^{(k-1)}\right)\frac{1}{4} \end{aligned}$$

- ☑ Implement the Jacobi iterative method for this set of equations.
- Your input parameters should be only
  - ☑ the total number of iterations,
  - ☑ the tolerance, as well as
  - ☑ n (which determines the size of the problem),

    i.e., there are no longer any $\bm{A}$ and $\bm{b}$ to be passed in.
- ☑ Choose a tolerance of 10^{-5} and print out the solution vectors for
  - ☑ $n = 10$ and for
  - ☑ $n = 20$,
  - ☑ each time also comparing with the output of `numpy.linalg.solve()`
    - ☑ To produce this, define $\bm{A}$ and $\bm{b}$ explicitly.

In [275…
```
''' INPUT PARAMETERS. Let these variables be
    k_max       the total number of iterations
    tol         tolerance value: usually 1E-8 but can be changed according
    n_dim       size of the problem (max number of dimensions)
'''
def convergence_pass(old_obj, new_obj, tolerance):
    # return (max(new_obj - old_obj) < tolerance)
    temp = abs((new_obj-old_obj)/new_obj)
    return max(temp) < tolerance

def convergence_calc(old_obj, new_obj):
    # return (max(new_obj - old_obj) < tolerance)
    return np.sum(abs((new_obj-old_obj)/new_obj))

def jacobi_spec(n_dim, k_max = 300, tol = 1E-8):
    #   Declare old and new arrays
    x_old = np.zeros(n_dim)
    x_new = np.zeros(n_dim)

    for k in range(k_max):
        #   First equation
        x_new[0] = 0.25*(1 + x_old[1] + x_old[-1])

        #   Second equation
        for i in range(1, n_dim-1):
            x_new[i] = 0.25*(1 + x_old[i-1] + x_old[i+1])

        #   Third equation
        x_new[-1] = 0.25*(1 + x_old[0] + x_old[-2])

        if convergence_pass(x_old, x_new, tol) == True:
            #print(f'{convergence_pass(x_old, x_new, tol)}')
            print(f'Converged after k = {k+1} iterations.')
            return x_new
        x_old = x_new.copy()

    return x_new
```

In [276…
```
#   We now supply arbitrary values given by the problem.
tol_arb = 1E-5
n_arb = np.array([10, 20])

results = [jacobi_spec(i, tol = tol_arb) for i in n_arb]

print(f'Given n = 10, solution to (A|b) is x = \n {results[0]} \n')
print(f'Given n = 20, solution to (A|b) is x = \n {results[1]}')
#[print(f'Given n = {i[0]}, vector x = \n {i[1]}') for i in zip(n_arb, re
```

```
Converged after k = 17 iterations.
Converged after k = 17 iterations.
Given n = 10, solution to (A|b) is x =
 [0.49999619 0.49999619 0.49999619 0.49999619 0.49999619 0.49999619
 0.49999619 0.49999619 0.49999619 0.49999619]

Given n = 20, solution to (A|b) is x =
 [0.49999619 0.49999619 0.49999619 0.49999619 0.49999619 0.49999619
 0.49999619 0.49999619 0.49999619 0.49999619 0.49999619 0.49999619
 0.49999619 0.49999619 0.49999619 0.49999619 0.49999619 0.49999619
 0.49999619 0.49999619]
```

In [282…
```
#   Next, we can then define A and b depending on some defined dimension
max_dims = n_arb

bs = [np.ones(i).reshape((i, 1)) for i in max_dims]     #   Vertical arra
bs = [np.ones(i) for i in max_dims]
#bs = np.ones(max_dims).reshape((max_dims, 1))
#bs = np.transpose(np.ones(max_dims))

A = [np.zeros((i, i)) for i in max_dims]

for i in A:
    np.fill_diagonal(i, 4)
    i[0,-1] = -1
    i[-1, 0] = -1

    #   I know that this can be done (inefficiently) with for-loops,
    #   but luckily for me this has been asked by someone before.
    r,c = np.indices(i.shape)
    i[r==c-1] = -1      #   Superdiagonal
    #i[r==c] = 4        #   Principal diagonal
    i[r==c+1] = -1      #   Subdiagonal
    #   Thanks to Saullo G.P. Castro who shared this solution in StackOve
    #   Source: https://stackoverflow.com/questions/18026541/make-special

#   xs can now be solved from the derived A and bs.
xs = [np.linalg.solve(i[0], i[1]) for i in zip(A, bs)]
```

In [292…
```
print(f'Given n = {n_arb[0]}, x = \n {x[0]} \n')
print(f'Given n = {n_arb[1]}, x = \n {x[1]} \n')
```

```
Given n = 10, x =
 [0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5]

Given n = 20, x =
 [0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5 0.5
 0.5 0.5]
```

> The difference of both methods is given in the code block below:

In [ ]:
```
deltas = [(xs[i] - results[i]) for i in range(len(n_arb))]

print(f'The average error at n = 10 is {np.mean(deltas[0]):.5g}')
print(f'The average error at n = 20 is {np.mean(deltas[1]):.5g}')
```

```
The average error at n = 10 is 3.8147e-06
The average error at n = 20 is 3.8147e-06
```

> Both situations get close to the NumPy function result, well below the set threshold $\epsilon =$ `1e-05`.